UT4. ALMACENAMIENTO EN LA NUBE

FICHA TÉCNICA DE LA UNIDAD

Resultados de Aprendizaje (RA)	**RA3**
Criterios de Evaluación (CE)	**RA3:** a, b, c, d, e, f
Tiempo	**Total**: 10 horas (**Teoría**: 2 h / Práctica: 8 h)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. EL PARADIGMA CLOUD COMPUTING Y MODELOS DE SERVICIO

- 2.1 Definición y evolución tecnológica
- 2.2 Modelos de Servicio: IaaS, PaaS, SaaS
- 2.3 Arquitecturas Multitenancy frente a Single-Tenant

3. SISTEMAS DE ALMACENAMIENTO WEB Y GESTORES DE ARCHIVOS

- 3.1 Tipos de almacenamiento en la nube: Bloques, Archivos y Objetos
- 3.2 Protocolos y APIs de acceso (RESTful, WebDAV)
- 3.3 Seguridad perimetral, Control de Accesos Basado en Roles (RBAC) y ACLs
- Ejemplo Práctico: Automatización de Copias de Seguridad con Scripting

4. SUITES DE OFIMÁTICA WEB Y TRABAJO COLABORATIVO

- 4.1 Análisis comparativo de infraestructura (Google Workspace vs. Microsoft 365)
- 4.2 Mecanismos de sincronización síncrona y resolución de conflictos de concurrencia
- 4.3 Auditoría de accesos y parsing de logs de operaciones

5. SERVICIOS WEB MULTIMEDIA Y STREAMING DE DATOS

- 5.1 Protocolos de streaming (HLS, DASH) y Codecs
- 5.2 Infraestructuras de Entrega de Contenidos (CDN)

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

El almacenamiento y la gestión de recursos de información han migrado desde las infraestructuras de escritorio locales hacia entornos centralizados y distribuidos en la red. Tradicionalmente, un técnico microinformático gestionaba la persistencia de datos mediante sistemas de archivos locales (NTFS, ext4) y suites ofimáticas nativas integradas directamente en el hardware del cliente. Sin embargo, los problemas de portabilidad, costes de licenciamiento, fragmentación de versiones y dificultades en la actualización centralizada han impulsado la adopción del *Cloud Computing*.

Relación Curricular: Esta unidad didáctica se conecta directamente con los conocimientos de infraestructuras de red del módulo de **Redes Locales** (configuración IP, enrutamiento, puertos), la administración de almacenamiento base del módulo de **Sistemas Operativos Monopuesto/Red** (permisos locales, cuotas de disco) y provee la base teórica operativa necesaria para la posterior implantación de plataformas de educación a distancia (Moodle) y la monitorización de servicios críticos en producción que se abordan en el resto del ciclo SMR.

2. EL PARADIGMA CLOUD COMPUTING Y MODELOS DE SERVICIO

2.1 Definición y evolución tecnológica

El *Cloud Computing* (Computación en la Nube) es el modelo tecnológico que permite el acceso omnipresente, adaptado y bajo demanda a un conjunto direccionable de recursos de computación configurables (redes, servidores, almacenamiento, aplicaciones y servicios) que pueden ser desplegados rápidamente con el mínimo esfuerzo de gestión o interacción con el proveedor del servicio.

2.2 Modelos de Servicio: IaaS, PaaS, SaaS

Los servicios en la nube se clasifican según el nivel de abstracción y responsabilidad técnica que asume el administrador del sistema:

- **IaaS (Infrastructure as a Service):** El proveedor ofrece abstracciones de hardware (máquinas virtuales, almacenamiento en bloque, direccionamiento de red virtualizado). El técnico instala, configura y mantiene el sistema operativo, los servicios middleware y las aplicaciones finales. *Ejemplos industriales: AWS EC2, Google Cloud Compute Engine.*
- **PaaS (Platform as a Service):** Se abstrae el sistema operativo y la pila de ejecución informática. El desarrollador o técnico interactúa únicamente con entornos de ejecución o contenedores de bases de datos. El proveedor gestiona el parcheado del núcleo de red y el escalado de hardware. *Ejemplos industriales: Heroku, AWS Elastic Beanstalk.*
- **SaaS (Software as a Service):** Abstracción completa de la infraestructura. El usuario o técnico interactúa con una aplicación web funcional consumible

directamente desde el navegador web mediante protocolos HTTP/HTTPS. *Ejemplos industriales: Google Docs, Microsoft 365, Nextcloud en modo gestionado.*

2.3 Arquitecturas Multitenancy frente a Single-Tenant

En entornos corporativos y de provisión SaaS, entender la arquitectura del backend determina las políticas de aislamiento y rendimiento:

- **Single-Tenant (Inquilino Único):** Cada cliente cuenta con una instancia de software dedicada independiente, un servidor web aislado y una base de datos propia. Ofrece máximo aislamiento de seguridad pero incrementa el coste operativo de mantenimiento.
- **Multitenant (Multi-inquilino):** Una única instancia lógica de la aplicación y un único motor de base de datos da servicio a múltiples clientes (inquilinos). Los datos de cada cliente se aíslan lógicamente mediante identificadores únicos (tenant_id) indexados en las tablas relacionales o esquemas lógicos segregados de la base de datos. Reduce los costes de infraestructura drásticamente y centraliza los backups globales.

3. SISTEMAS DE ALMACENAMIENTO WEB Y GESTORES DE ARCHIVOS

3.1 Tipos de almacenamiento en la nube: Bloques, Archivos y Objetos

La persistencia de datos en entornos Cloud difiere estructuralmente según su acceso y optimización:

- **Almacenamiento en Bloque:** Los datos se dividen en bloques del mismo tamaño sin metadatos adjuntos y se gestionan como si fuesen discos físicos locales independientes expuestos a través de redes de área de almacenamiento (SAN). Ideal para sistemas operativos y motores de bases de datos de alto rendimiento.
- **Almacenamiento de Archivos (File Storage):** Organizado bajo una estructura jerárquica de directorios y subdirectorios tradicional utilizando protocolos de red locales compartidos como NFS (Network File System) o SMB/CIFS.
- **Almacenamiento de Objetos (Object Storage):** Estructura plana donde cada archivo se convierte en un objeto que contiene el flujo binario del archivo, un identificador único global (UUID) y metadatos detallados extensibles. No utiliza rutas de directorios tradicionales; el acceso se realiza mediante llamadas directas HTTP y APIs RESTful (verbos GET, POST, PUT, DELETE). Es la base operativa de los buckets modernos.

3.2 Protocolos y APIs de acceso (RESTful, WebDAV)

Los gestores de archivos web independientes integran mecanismos de interoperabilidad nativos para que los sistemas de escritorio locales mapeen las carpetas remotas:

- **WebDAV (Web-based Distributed Authoring and Versioning):** Extensión del protocolo HTTP que permite a los clientes realizar operaciones de edición, movimiento, creación y bloqueo de archivos en un servidor web remoto, actuando como un sistema de archivos compartido tradicional sobre el puerto estándar 80/443.

3.3 Seguridad perimetral, Control de Accesos Basado en Roles (RBAC) y ACLs

El aseguramiento técnico del almacenamiento descentralizado exige la implementación rigurosa de:

1. **RBAC (Role-Based Access Control):** Asignación de permisos (Lectura, Escritura, Eliminación) basados explícitamente en el perfil laboral del usuario (Administrador, Editor, Invitado) y no de forma individualizada.
2. **Cifrado de datos:** Implementación obligatoria de algoritmos simétricos TLS 1.3 para mitigar ataques *Man-in-the-Middle* en el tránsito de los ficheros de datos, y cifrado AES-256 en reposo sobre los discos duros de almacenamiento físicos.

Ejemplo Práctico: Automatización de Copias de Seguridad con Scripting

Un técnico microinformático debe automatizar los procesos de sincronización hacia el sistema de almacenamiento central de la empresa. A continuación se presenta un script en Bash documentado para empaquetar de forma incremental un directorio local y cargarlo en un servidor de almacenamiento web mediante la API de sincronización.

```
#!/bin/bash
#
================================================================================
# SCRIPT DE AUTOMATIZACIÓN DE BACKUPS CORPORATIVOS E INTEGRACIÓN
CLOUD
# PROCESO: Empaquetado comprimido y sincronización hacia repositorio
remoto
#
================================================================================
# Definición de variables de entorno locales
ORIGEN="/var/www/html/datos_empresa"
DESTINO_TEMPORAL="/tmp/backup_corporativo_$(date +%Y%m%d).tar.gz"
URL_STORAGE_CLOUD="https://api.storagecloud.local/v1/buckets/respaldos/objetos"
TOKEN_AUTENTICACION="Bearer xX_TokenSeguro_SMR_2026_Xx"
# Paso 1: Comprobación de la integridad del directorio origen
if [ ! -d "$ORIGEN" ]; then
```

```
echo "[ERROR] - $(date +'%T'): El directorio de datos origen no
existe. Abortando." >> /var/log/backup_cloud.log
exit 1
fi
# Paso 2: Generación del archivo empaquetado y comprimido en formato
tar.gz
echo "[INFO] - Iniciando compresión de archivos..."
tar -czf "$DESTINO_TEMPORAL" -C "$ORIGEN" .
# Paso 3: Transmisión del objeto binario hacia el almacenamiento en
la nube vía API RESTful
echo "[INFO] - Transmitiendo backup de forma segura al almacenamiento
Cloud..."
RESPUESTA_HTTP=$(curl -s -o /dev/null -w "%{http_code}" -X PUT \
-H "Authorization: $TOKEN_AUTENTICACION" \
-H "Content-Type: application/gzip" \
--data-binary @"$DESTINO_TEMPORAL" \
"$URL_STORAGE_CLOUD/backup_$(date +%Y%m%d).tar.gz")
# Paso 4: Validación del código de estado HTTP retornado por el
servidor remoto
if [ "$RESPUESTA_HTTP" -eq 201 ] || [ "$RESPUESTA_HTTP" -eq 200 ];
then
echo "[OK] - $(date +'%T'): Backup transmitido correctamente con
código $RESPUESTA_HTTP" >> /var/log/backup_cloud.log
else
echo "[CRÍTICO] - $(date +'%T'): Fallo de persistencia en la nube.
HTTP Código: $RESPUESTA_HTTP" >> /var/log/backup_cloud.log
fi
# Paso 5: Limpieza del entorno local (Remoción del archivo
temporal)
rm -f "$DESTINO_TEMPORAL"
exit 0
```

4. SUITES DE OFIMÁTICA WEB Y TRABAJO COLABORATIVO

4.1 Análisis comparativo de infraestructura (Google Workspace vs. Microsoft 365)

Dimensión Técnica	Google Workspace (Docs/Sheets)	Microsoft 365 Online
Arquitectura Base	Nativa web. Diseño desacoplado orientado a microservicios optimizado para la ejecución ligera en navegadores.	Enfoque híbrido. Motor adaptado desde código nativo de escritorio (Win32) y traducido para compatibilidad web.
Formato de Archivo Nativo	Almacenamiento basado en JSON estructurado indexado en bases de datos internas propietarias.	Estructuras basadas en el estándar comprimido OpenXML (formatos nativos .docx, .xlsx, .pptx).
Control de Concurrencia	Algoritmos de Transformación Operacional (OT) ejecutados asíncronamente en tiempo real de baja latencia.	Bloqueo lógico por párrafos o celdas con sincronización mediante Diff/Patch estructurados sobre OneDrive/SharePoint.

4.2 Mecanismos de sincronización síncrona y resolución de conflictos de concurrencia

Cuando múltiples usuarios editan de forma simultánea la misma sección de una hoja de cálculo o procesador de textos web, la aplicación web no bloquea todo el documento. Utiliza el modelo de **Transformación Operacional (OT)**: las ediciones realizadas de forma local por un cliente se capturan como "operaciones de inserción o eliminación de caracteres" con coordenadas relativas. Estas operaciones se envían al servidor en segundo plano, el cual reescribe y transforma de forma dinámica los índices posicionales de las peticiones de los otros usuarios concurrentes para garantizar que el documento final mantenga la consistencia global.

4.3 Auditoría de accesos y parsing de logs de operaciones

La monitorización de las acciones de los usuarios dentro de las herramientas de almacenamiento web corporativas es una tarea crítica de mantenimiento informático preventivo para evitar fugas de información confidencial.

Ejemplo Práctico: Análisis y parsing automático de logs de almacenamiento web

Un administrador debe procesar los logs en crudo generados por un servidor de almacenamiento de archivos web para identificar e interceptar peticiones de borrado no autorizadas ejecutadas por usuarios del sistema.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
#
================================================================================
# SCRIPT DE AUTOMATIZACIÓN DE AUDITORÍA: PARSING DE LOGS DE
ALMACENAMIENTO WEB
# OBJETIVO: Detectar operaciones críticas de eliminación de archivos
en la nube
#
================================================================================
import re
import sys
# Simulación de líneas de registro en crudo extraídas de un servidor
cloud
registros_log = [
'2026-06-16 08:12:04 USER:juan.perez IP:192.168.1.45 OP:READ
FILE:/documentos/informe.pdf STATUS:200',
'2026-06-16 08:14:22 USER:ana.gomez IP:192.168.1.102 OP:WRITE
FILE:/imagenes/logo.png STATUS:201',
'2026-06-16 08:30:15 USER:juan.perez IP:192.168.1.45 OP:DELETE
FILE:/contabilidad/facturas_2025.zip STATUS:204',
'2026-06-16 09:01:10 USER:invitado_red IP:10.0.0.12 OP:DELETE
FILE:/backup/sistema_antiguo.iso STATUS:403'
]
# Expresión regular estructurada para capturar campos del formato de
log estándar
patron_log = re.compile(
r'(?P<fecha>\d{4}-\d{2}-\d{2})\s(?P<hora>\d{2}:\d{2}:\d{2})\s'
r'USER:(?P<usuario>\S+)\sIP:(?P<ip>\S+)\s'
r'OP:(?P<operacion>\S+)\sFILE:(?P<archivo>\S+)\sSTATUS:(?P<estado>\d+)'
)
print("================================================================================")
print(" REPORTE DE AUDITORÍA: ELIMINACIÓN DE ARCHIVOS ")
print("================================================================================")
# Procesamiento línea a línea del flujo de datos
for linea in registros_log:
coincidencia = patron_log.match(linea)
if coincidencia:
datos = coincidencia.groupdict()
# Filtrado técnico por operación de eliminación crítica (DELETE)
if datos['operacion'] == 'DELETE':
print(f"ALERTA CRÍTICA detectada a las {datos['hora']}:")
print(f" - Operario responsable: {datos['usuario']}")
print(f" - Dirección IP origen: {datos['ip']}")
print(f" - Recurso afectado: {datos['archivo']}")
# Evaluación de la autorización según el código de estado HTTP
retornado
if datos['estado'] == '204':
```

```
print(" - Estado de la acción: [ÉXITO - ARCHIVO ELIMINADO]")
elif datos['estado'] == '403':
print(" - Estado de la acción: [DENEGADO - INTENTO DE VIOLACIÓN DE
ACCESO]")
print("-" * 70)
```

5. SERVICIOS WEB MULTIMEDIA Y STREAMING DE DATOS

5.1 Protocolos de streaming (HLS, DASH) y Codecs

El alojamiento multimedia en plataformas web no funciona descargando por completo los archivos binarios de audio y vídeo a la máquina local del cliente debido al coste inasumible de ancho de banda. En su lugar, se utilizan arquitecturas de streaming basado en HTTP:

- **HLS (HTTP Live Streaming) & MPEG-DASH:** Tecnologías que funcionan fragmentando el archivo de vídeo original en pequeños bloques segmentados (típicamente archivos de 2 a 10 segundos de duración) y generando un archivo de índice estructurado en formato de texto plano (extensión .m3u8 en HLS o .mpd en DASH). El reproductor web del navegador descarga secuencialmente los fragmentos indexados de manera dinámica basándose en el ancho de banda disponible en tiempo real en la red del cliente (Streaming de Tasa de Bits Adaptativa).

5.2 Infraestructuras de Entrega de Contenidos (CDN)

Para minimizar la latencia provocada por la distancia geográfica respecto al servidor web de almacenamiento centralizado, las empresas implementan una **CDN (Content Delivery Network)**. Se trata de una red geográficamente distribuida de servidores proxy de caché (denominados servidores de borde o *Edge Servers*). Cuando el navegador de un cliente solicita un objeto multimedia estático o un fragmento de vídeo, el sistema DNS intercepta la resolución del dominio y enruta la conexión hacia el servidor de borde físicamente más cercano al cliente, reduciendo drásticamente los tiempos de carga y el consumo del enlace principal del servidor origen.